

JobFinder

AI Agent for smarter job search

FastAPI

Next.js

ReAct Agent

Railway + Vercel



resume
→ agent
→ ranked jobs

Problem

Manual job search is slow, noisy, and hard to compare with a resume.

Too many irrelevant roles

Search results mix junior jobs with senior, director, sales, legal, and unrelated positions.

No fit explanation

Candidates must manually compare requirements, skills, seniority level, location, and job description.

Time-consuming workflow

A user may open dozens of vacancies before finding a few realistic opportunities.

Target users: junior/middle AI, GenAI, Python, backend, and full-stack candidates.

Solution

An autonomous agent that analyzes the resume and ranks relevant jobs.

1. Resume analysis

Extracts skills, tools, experience level, location, and target role.

2. Multi-source search

Uses custom tools to search hh.kz, LinkedIn-style results, RemoteOK, and We Work Remotely.

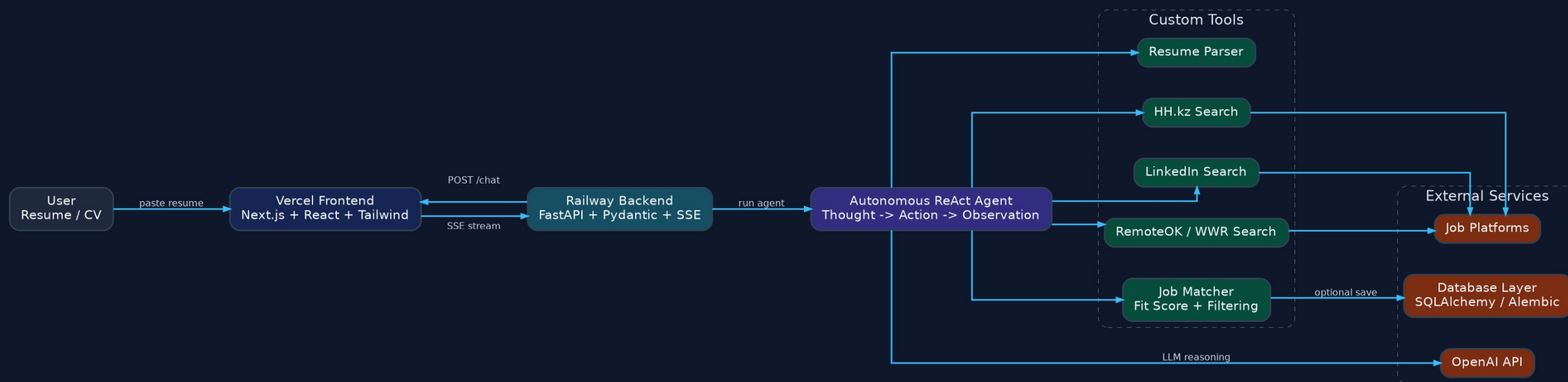
3. Ranking & filtering

Removes unrelated jobs and scores each vacancy by fit with the candidate profile.

Output: ranked vacancies with match score, company, source, description, and explanation.

Architecture

End-to-end deployed AI application.



Deployment

Backend runs on Railway. Frontend runs on Vercel. Docker and docker-compose are included for local reproducibility.

API

FastAPI exposes GET /health and POST /chat with Pydantic validation and streaming events.

Agent & Custom Tools

The ReAct loop decides the next step instead of using a rigid workflow.

ReAct Agent

Thought -> Action -> Observation -> Final Answer. The agent can choose parsing, search, and ranking tools.

Custom tools

resume parser, hh.kz search, LinkedIn search, RemoteOK search, WWR search, matching and filtering.

Quality guardrails

Focused search queries, seniority-aware filtering, irrelevant role filtering, match score, and reason generation.

Production Readiness

Core capstone requirements are covered.

Backend

FastAPI, Pydantic validation, logging, rate limiting, streaming response, Swagger docs.

Frontend

Next.js web UI with resume input, loading states, streamed result handling, and vacancy cards.

Deployment

Dockerfile, docker-compose.yml, Railway backend config, Vercel frontend config.

Testing

Pytest covers root endpoint, health endpoint, chat validation, and job matching filters.

Results & Next Steps

A working portfolio-ready AI product with a clear improvement path.

Delivered

Live frontend, live backend, autonomous agent, custom tools, deployment config, docs, tests, architecture, and presentation.

Future improvements

User accounts, PDF/DOCX upload in web UI, saved jobs, cover letter generation, PostgreSQL production database, LangSmith observability.

JobFinder helps candidates move from noisy search results to focused, explainable job recommendations.